

You're getting this error because of how **unbuffered channels** work in Go.

What's happening in your code

```
messageChannel := make(chan string)
```

```
messageChannel <- "ping" // ❌ send  
message := <-messageChannel // ❌ receive
```

You created an **unbuffered channel**, which means:

- A send (<-) will **block until someone receives**
- A receive will **block until someone sends**

So when your program reaches this line:

```
messageChannel <- "ping"
```

Go tries to send "ping", but there is **no goroutine receiving yet**.
So it waits forever.

Then the next line (receive) is never reached → Go detects:

"all goroutines are asleep" → deadlock

Why Go crashes with deadlock

Your **main** goroutine is the only one running.
It is stuck waiting on itself.

So Go says:

"Nobody is running, everyone is waiting → deadlock."

Fix options

✅ 1. Use a goroutine (most common fix)

```
package main
```

```
import "fmt"

func main() {
    messageChannel := make(chan string)

    go func() {
        messageChannel <- "ping"
    }()

    message := <-messageChannel
    fmt.Println(message)
}
```

Now:

- Sender runs in background
 - Receiver waits safely
-

✓ 2. Send first only after receiver is ready (reorder with goroutine still needed)

You *cannot* fix this just by swapping lines in the same goroutine.

This will still deadlock:

```
message := <-messageChannel
messageChannel <- "ping"
```

✓ 3. Use a buffered channel

```
messageChannel := make(chan string, 1)
```

```
messageChannel <- "ping"
message := <-messageChannel
fmt.Println(message)
```

Now:

- Channel can store 1 value without blocking send

Key idea to remember

Unbuffered channels require sender and receiver to meet at the same time.

If they don't, Go blocks → and if everyone blocks → deadlock.